North Carolina_County_Level_Poverty_Analysis

Waleed Abdulla

2025-04-04

1 Data Preparation

Note that the following libraries are uploaded in the background to create a cleaner markdown file: knitr, fpp3, eeptools, usmap, stringr, urca

1.1 SAIPE data

```
NCRaw <- read.csv("Data/SAIPE_04-04-2025.csv") %>%
  select(Year, ID, Name, Poverty.Universe, Number.in.Poverty) %>%
  rename(fips = ID, `County Population` = Poverty.Universe, `Number in Poverty` = Number.in.Poverty)

#decomma() found in eeptools library transforms string numbers with commas into numeric values
NC <- NCRaw %>% mutate(`County Population` = decomma(`County Population`),
                       `Number in Poverty` = decomma(`Number in Poverty`)) %>% na.omit()
```

This project's state name, abbreviation, and FIPS code is North Carolina, NC, and 37, respectively.

```
NC %>% select(Name) %>% unique() %>% count()
```

```
##     n
## 1 100
```

```
NC %>% group_by(Name) %>% summarize(Max = max(`County Population`)) %>% arrange(desc(Max)) %>% head(5)
```

```
## # A tibble: 5 x 2
##   Name                   Max
##   <chr>                <dbl>
## 1 Wake County        1173780
## 2 Mecklenburg County 1145365
## 3 Guilford County     524468
## 4 Forsyth County      380940
## 5 Cumberland County   320926
```

There are 100 counties. The largest county ever recorded was Wake County with 1,173,780 people.

```
NC %>% filter(Year == 2023) %>% arrange(desc(`County Population`)) %>% head(9)
```

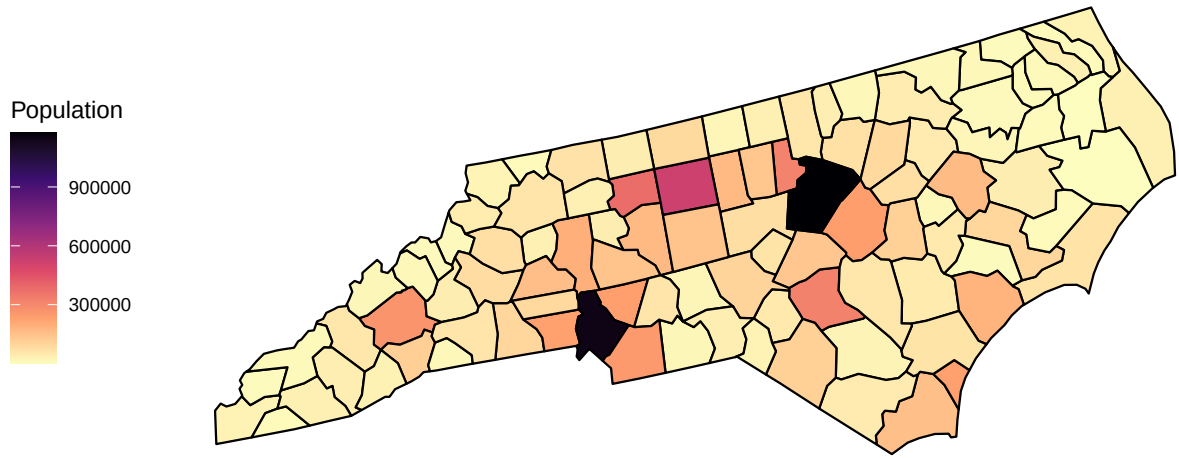
```
##   Year  fips               Name County Population Number in Poverty
## 1 2023 37183        Wake County           1173780             85051
## 2 2023 37119 Mecklenburg County           1145365            116447
```

```
## 3 2023 37081    Guilford County            524468             77550
## 4 2023 37067     Forsyth County            380940             56249
## 5 2023 37051  Cumberland County            320926             49047
## 6 2023 37063      Durham County            320369             33994
## 7 2023 37021    Buncombe County            269143             34611
## 8 2023 37179       Union County            252997             21166
## 9 2023 37101    Johnston County            239437             25952
```

```
topNine <- NC %>% filter(Year == 2023) %>% arrange(desc(`County Population`)) %>% head(9) %>% pull(Name)
```

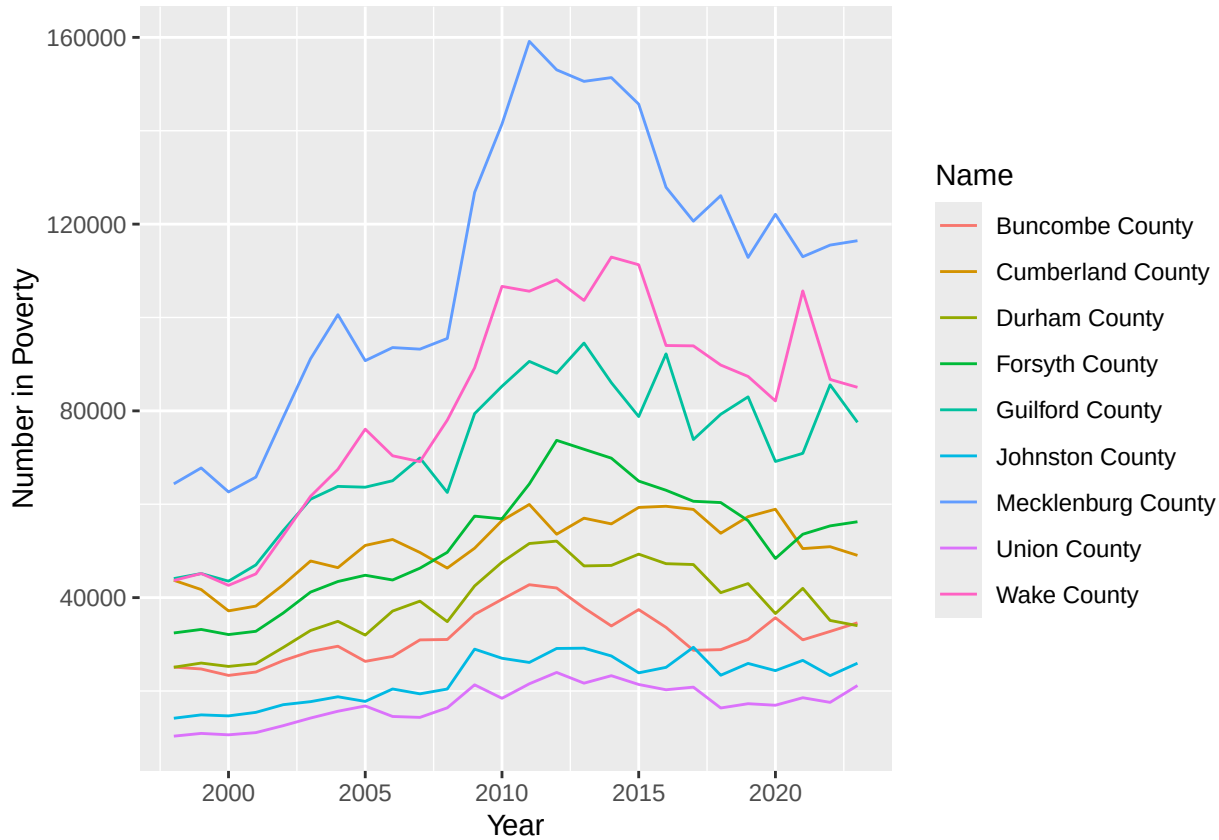
The above are the nine largest counties by the most recent year in the dataset (2023).

```
NCMap <- NC %>% filter(Year == 2023) %>% mutate(values = `County Population`, fips = fips) %>% select(fi
plot_usmap(regions = "counties", include = "North Carolina", data = NCMap) +
  scale_fill_viridis_c(option = "A", direction = -1) + labs(fill = "Population") + theme(legend.positio
```




```
NCGIS <- NC %>% filter(Year == 2023)
write.csv(NCGIS, "Data/North_Carolina_Counties_and_Poverty.csv")
```

```
NC9 <- NC %>% filter(Name %in% topNine)
NC9 %>% ggplot(aes(x=Year,y=`Number in Poverty`,colour = Name)) + geom_line()
```

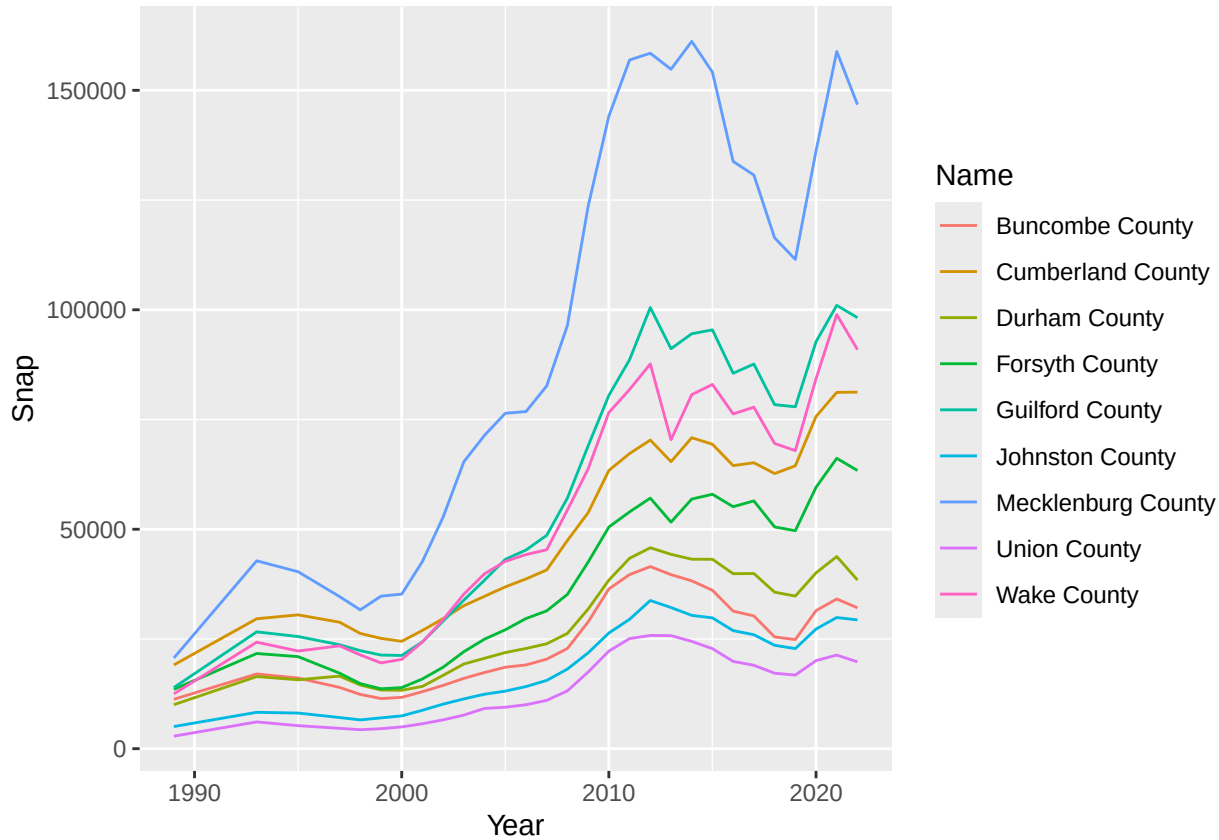


1.2 County SNAP Benefits

```
SNAPRaw <- read.csv("Data/cntysnap.csv", skip=5) %>%
  filter(State.FIPS.code == 37, County.FIPS.code != 0) %>%
  mutate(fips = paste(as.character(State.FIPS.code), sprintf("%03d", County.FIPS.code), sep = "") %>% as
  select(-c(County.FIPS.code, State.FIPS.code))

Snap <- SNAPRaw %>% pivot_longer(!c(Name,fips), names_to = "Year", values_to = "Snap") %>%
  mutate(Year = str_remove(Year,"Jul.") %>% as.integer(),
         Name = str_remove(Name,", NC"), Snap = decomma(Snap) %>% as.integer())

Snap %>% head(5)
```

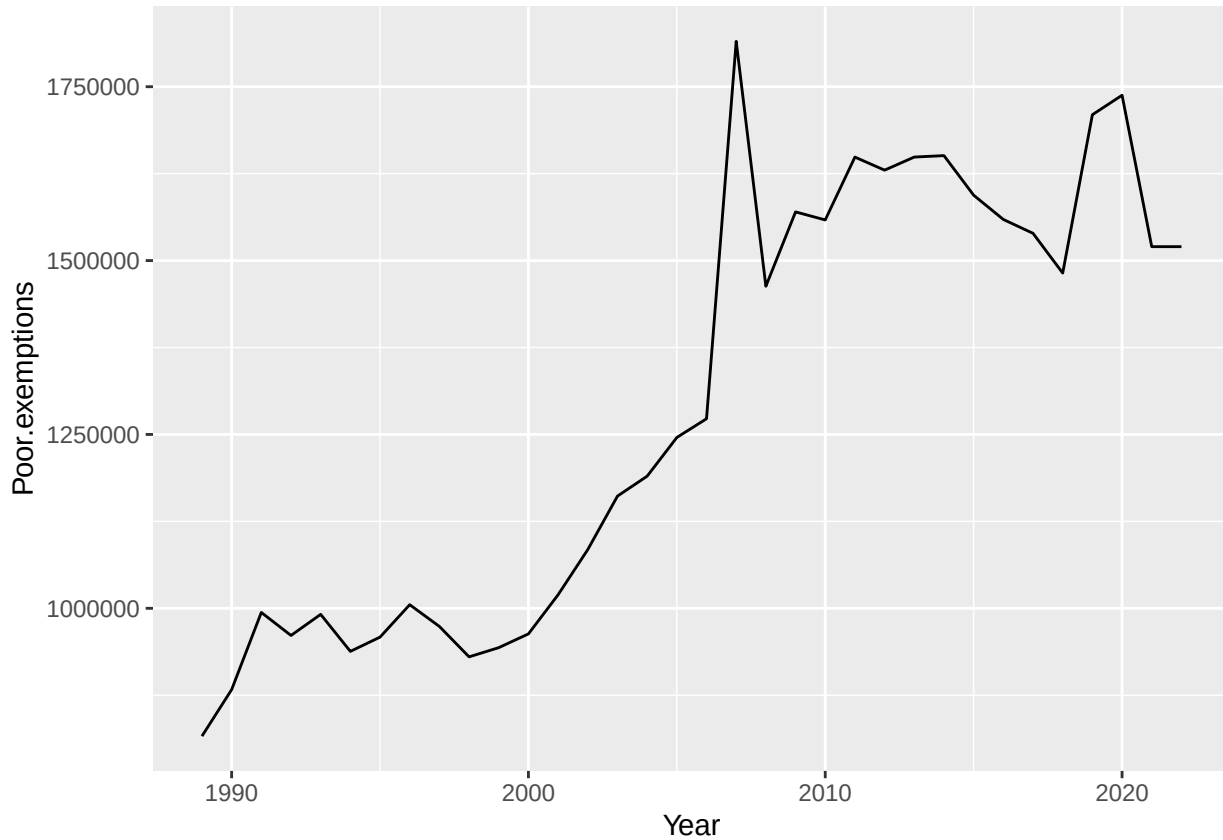
```
## # A tibble: 5 x 4
##   Name              fips  Year  Snap
##   <chr>            <int> <int> <int>
## 1 Alamance County 37001  2022 26993
## 2 Alamance County 37001  2021 28182
## 3 Alamance County 37001  2020 25794
## 4 Alamance County 37001  2019 22187
## 5 Alamance County 37001  2018 22251
```

```
Snap9 <- Snap %>% filter(Name %in% topNine)
Snap9 %>% ggplot(aes(x=Year,y=Snap,colour = Name)) + geom_line()
```



1.3 State IRS Data

```
IrsRaw <- read.csv("Data/irs.csv",skip=5)

Irs <- IrsRaw %>% filter(State.FIPS.code == 37) %>% mutate(Poor.exemptions = decomma(Poor.exemptions), 
  select(fips, Poor.exemptions, Year)
Irs %>% ggplot(aes(x=Year,y=Poor.exemptions)) + geom_line()
```



1.4 Merging the data

```
NCaroR <- left_join(Snap,Irs,by="Year")
NCaroR <- left_join(NCaroR,NC)
```

```
## Joining with `by = join_by(Name, Year)`
```

```
NCaro <- NCaroR %>% mutate(fips = fips.x) %>% select(-c(fips.y,fips.x)) %>%
  filter(Year > 1997) %>% select(Year, fips, Name, `Number in Poverty`, `County Population`, Snap, Poor
NCaro %>% head()
```

```
## # A tibble: 6 x 7
##    Year  fips Name            `Number in Poverty` `County Population`  Snap
##   <int> <int> <chr>                         <dbl>               <dbl> <int>
## 1  2022 37001 Alamance County               21639              170775 26993
## 2  2021 37001 Alamance County               20916              167890 28182
## 3  2020 37001 Alamance County               24978              165090 25794
## 4  2019 37001 Alamance County               23890              163253 22187
## 5  2018 37001 Alamance County               21661              160487 22251
## 6  2017 37001 Alamance County               22721              158154 23431
## # i 1 more variable: Poor.exemptions <dbl>
```

Now check for any problems

```r
NCaro$fips %>% unique()
```

```
##   [1] 37001 37003 37005 37007 37009 37011 37013 37015 37017 37019 37021 37023
##  [13] 37025 37027 37029 37031 37033 37035 37037 37039 37041 37043 37045 37047
##  [25] 37049 37051 37053 37055 37057 37059 37061 37063 37065 37067 37069 37071
##  [37] 37073 37075 37077 37079 37081 37083 37085 37087 37089 37091 37093 37095
##  [49] 37097 37099 37101 37103 37105 37107 37109 37111 37113 37115 37117 37119
##  [61] 37121 37123 37125 37127 37129 37131 37133 37135 37137 37139 37141 37143
##  [73] 37145 37147 37149 37151 37153 37155 37157 37159 37161 37163 37165 37167
##  [85] 37169 37171 37173 37175 37177 37179 37181 37183 37185 37187 37189 37191
##  [97] 37193 37195 37197 37199
```

```r
NCaro %>% group_by(fips) %>% summarize(count = n()) %>% filter(count != 25)
```

```
## # A tibble: 0 x 2
## # i 2 variables: fips <int>, count <int>
```

All 100 counties have 25 rows, from 2023 to 1998. There are only 100 unique counties in the dataset too.

```r
NCaro %>% is.na() %>% mean()
```

```
## [1] 0
```

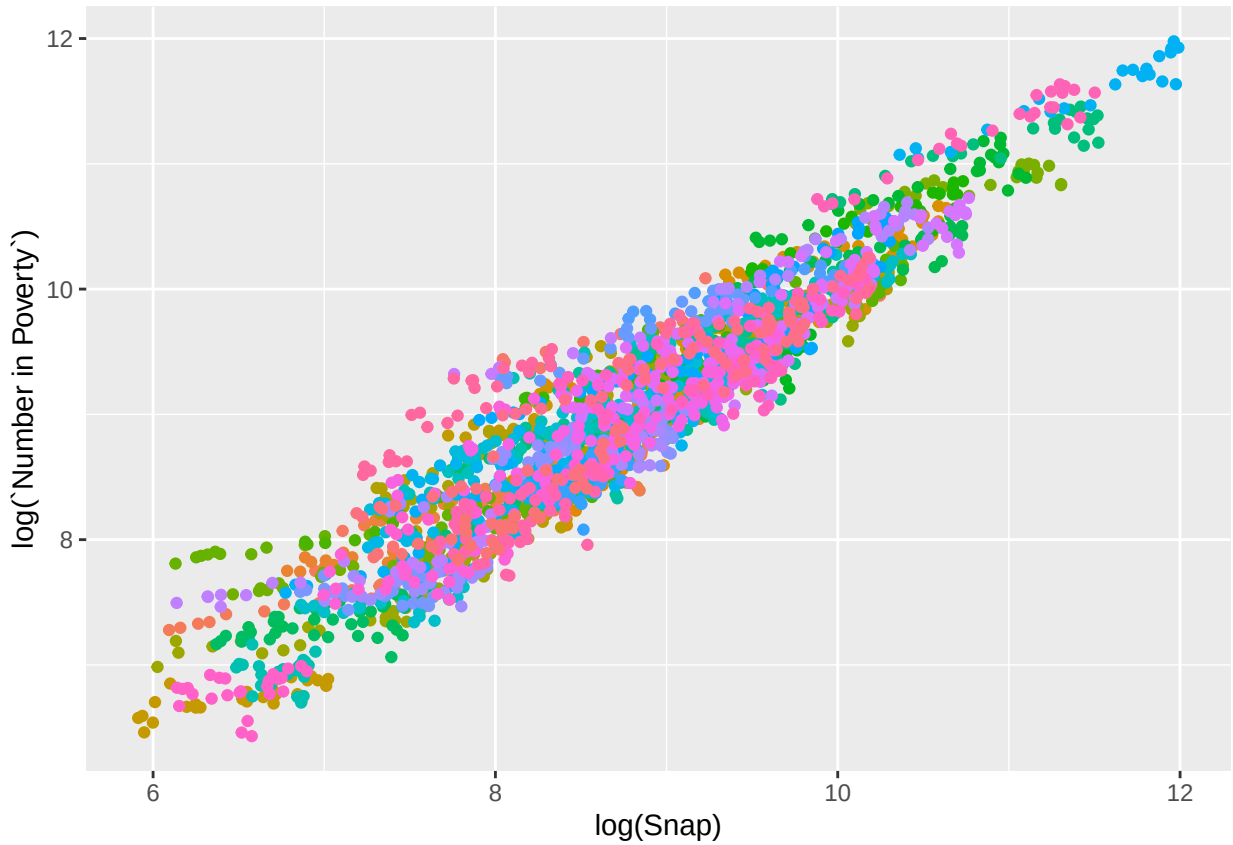
There are no NA values (removed in beginning).

After checking the county changes website, only two North Carolina Counties had substantial boundary changes where, in the 1990s, Carteret County received five square miles and lost 16 square miles from Craven County.

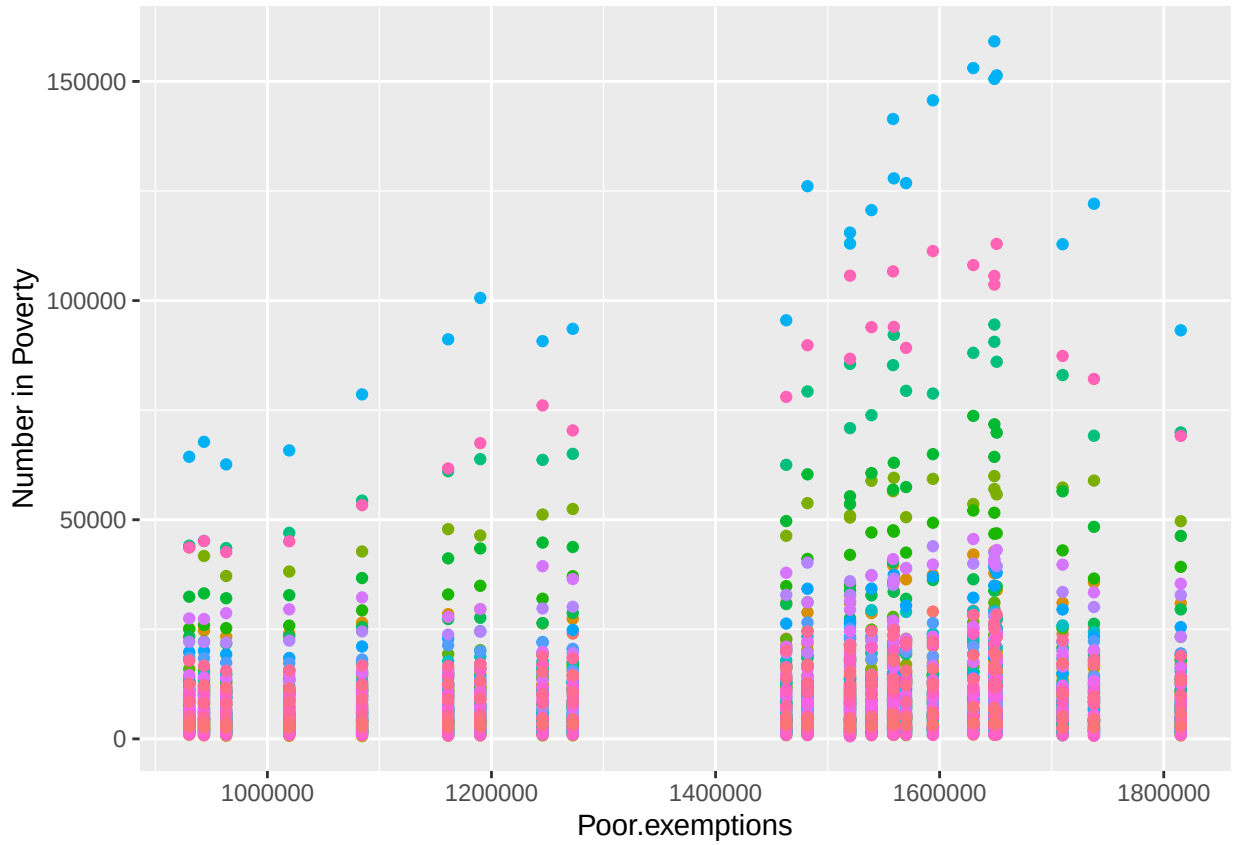
```r
NCTS <- NCaro %>% as_tsibble(Year, key=c(fips,Name))
NCTS %>% head()
```

```
## # A tsibble: 6 x 7 [1Y]
## # Key:       fips, Name [1]
##    Year  fips Name            `Number in Poverty` `County Population`  Snap
##   <int> <int> <chr>                         <dbl>               <dbl> <int>
## 1  1998 37001 Alamance County               11731              120061  3877
## 2  1999 37001 Alamance County               12588              128691  3114
## 3  2000 37001 Alamance County               12284              131098  3149
## 4  2001 37001 Alamance County               13339              132706  4003
## 5  2002 37001 Alamance County               14437              134359  4990
## 6  2003 37001 Alamance County               15377              135779  6191
## # i 1 more variable: Poor.exemptions <dbl>
```

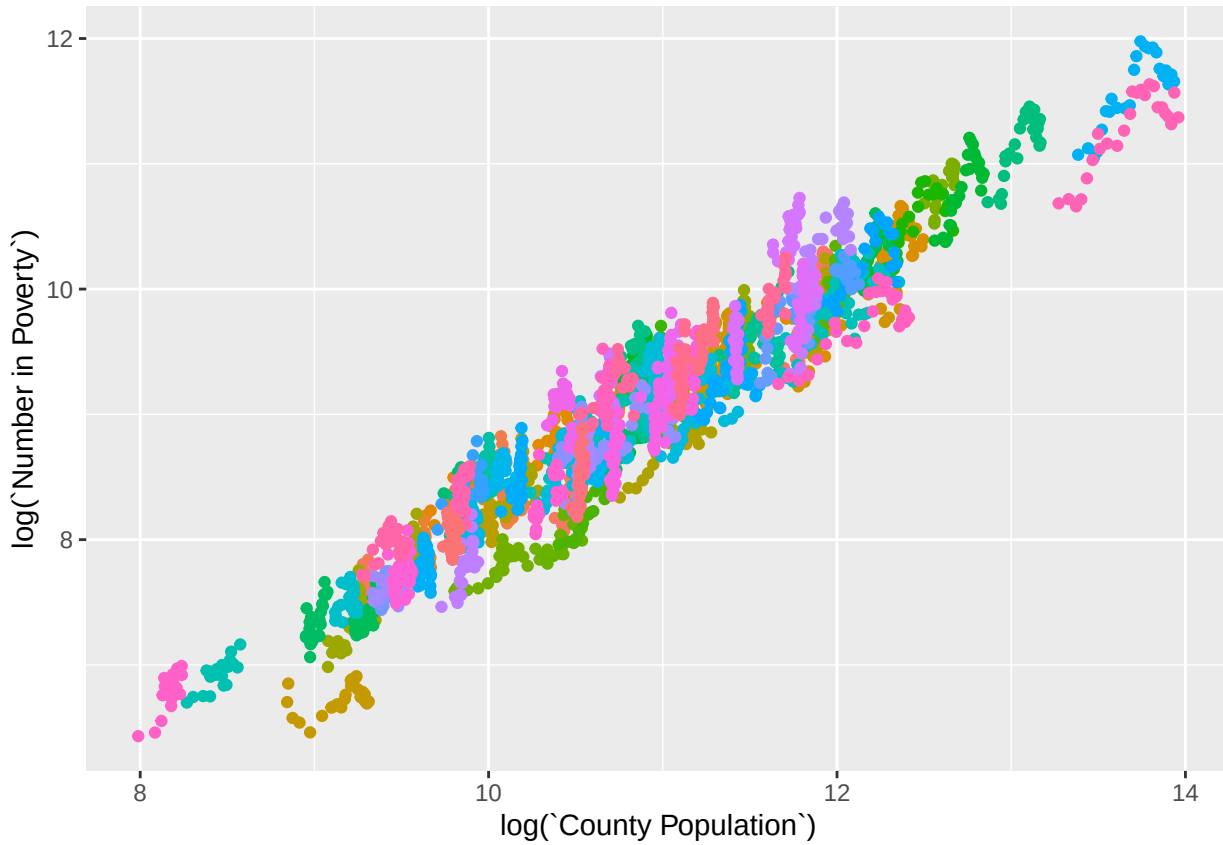
```r
NCTS %>% ggplot(aes(x=log(Snap),y=log(`Number in Poverty`), color = Name)) + geom_point(show.legend = F
```




```
NCTS %>% ggplot(aes(x=Poor.exemptions,y=`Number in Poverty`, color = Name)) + geom_point(show.legend = F
```




```
NCTS %>% ggplot(aes(x=log(`County Population`),y=log(`Number in Poverty`), color = Name)) + geom_point(s
```



Part 2: Linear Models

2.1 Variable Selection

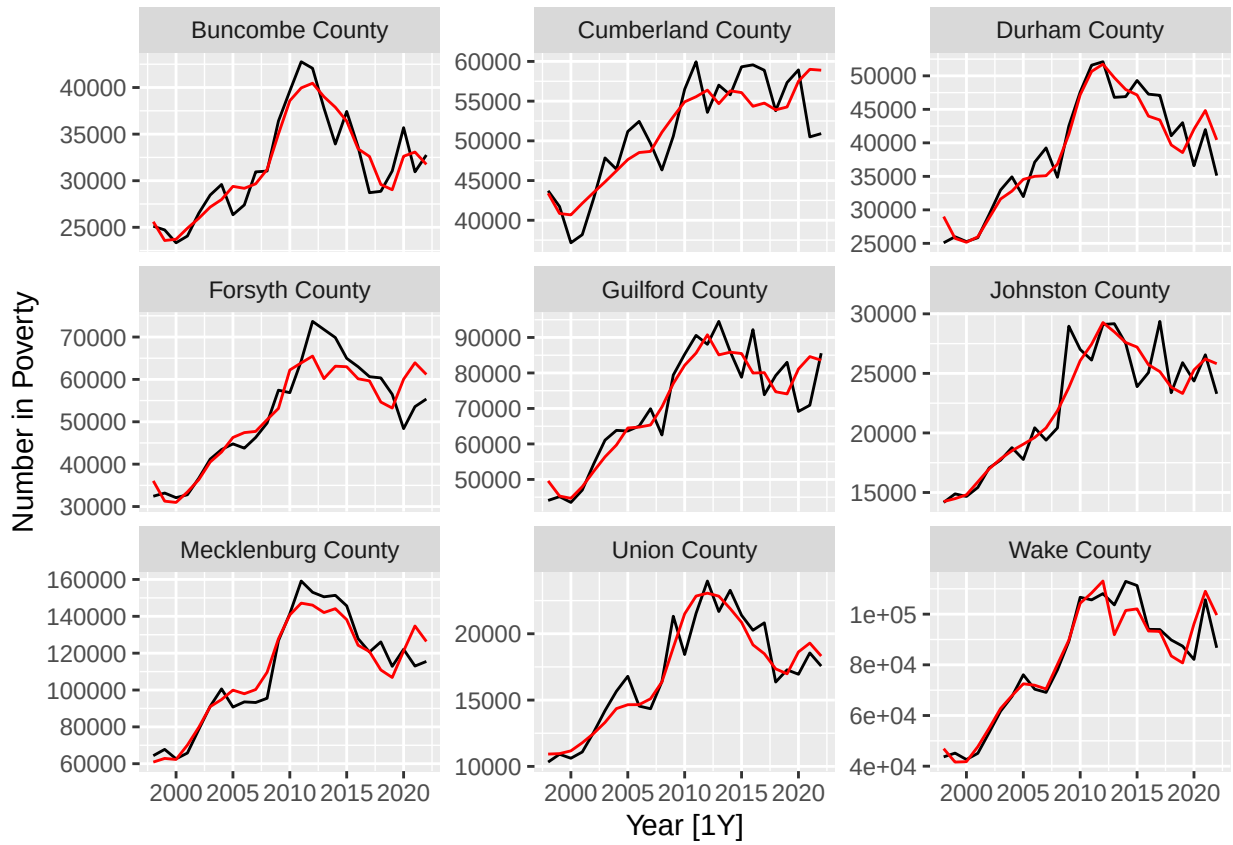
```
fitSet <- NCTS %>%
  model(
    P = TSLM(log(`Number in Poverty`) ~ log(`County Population`)),
    PS = TSLM(log(`Number in Poverty`) ~ log(`County Population`) + log(Snap)),
    PSE = TSLM(log(`Number in Poverty`) ~ log(`County Population`) + log(Snap) + log(Poor.exemptions)),
    S = TSLM(log(`Number in Poverty`) ~ log(Snap)),
    SE = TSLM(log(`Number in Poverty`) ~ log(Snap) + log(Poor.exemptions)),
    E = TSLM(log(`Number in Poverty`) ~ log(Poor.exemptions)),
    ES = TSLM(log(`Number in Poverty`) ~ log(`County Population`) + log(Poor.exemptions))
    )

glance(fitSet) %>% group_by(.model) %>% summarize(AIC = mean(AIC))
```

```
## # A tibble: 7 x 2
##   .model    AIC
##   <chr>   <dbl>
```

```
## 1 E      -104.
## 2 ES     -107.
## 3 P       -97.9
## 4 PS     -120.
## 5 PSE    -119.
## 6 S      -115.
## 7 SE     -114.
```

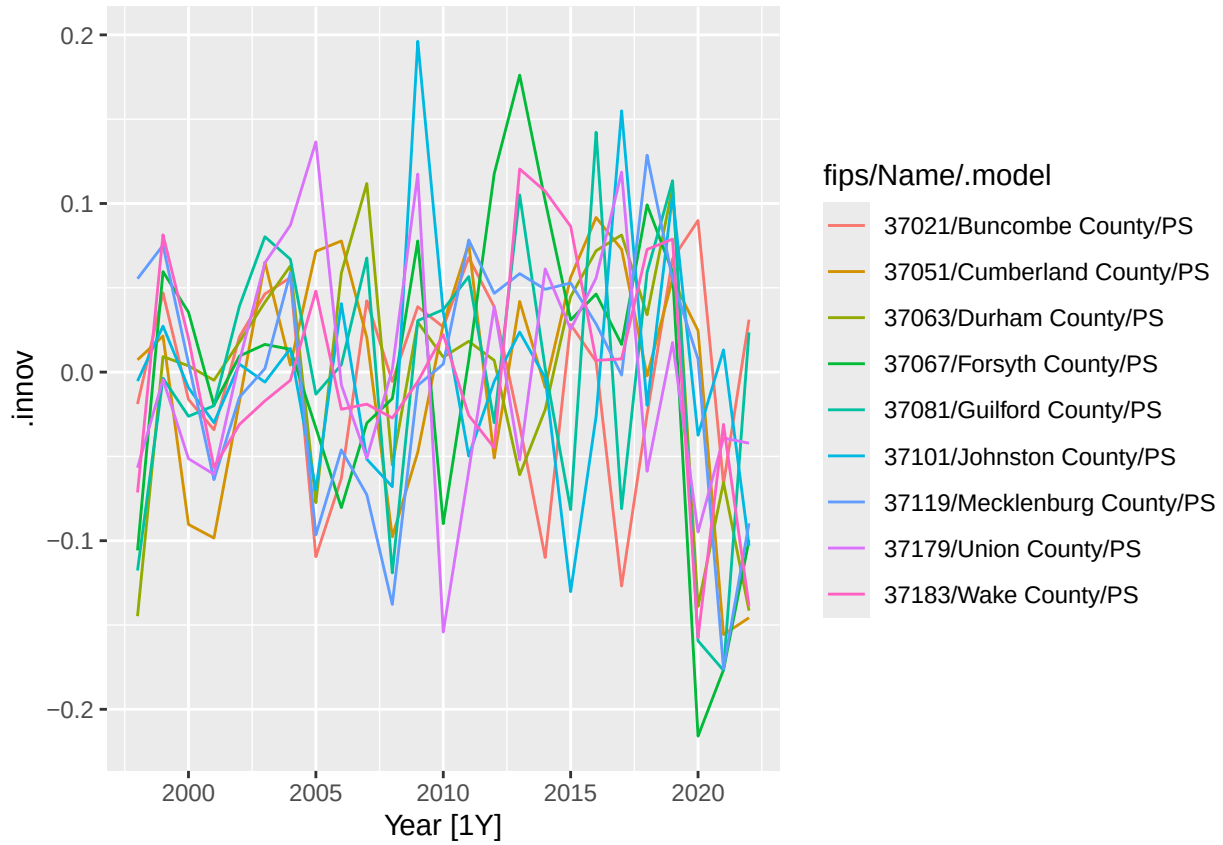
Based on the AIC test, the model that uses the Population and Snap variables does the best across all counties by a small margin.

```
NCTS %>% filter(Name %in% topNine) %>% autoplot(`Number in Poverty`, color = "black") +
  facet_wrap("Name", scales = "free_y") +
  geom_line(data = fitSet %>% fitted() %>% filter(.model == "PS", Name %in% topNine), aes(y = .fitted),
```



2.2 Residual Analysis

```
fitSet %>% filter(Name %in% topNine) %>% select(PS) %>% augment() %>% autoplot(.innov)
```




```
fitSet %>% filter(Name %in% topNine) %>% select(PS) %>% augment() %>%
  features(.innov, ljung_box, lag = 10) %>% filter(lb_pvalue <= 0.05)
```

```
## # A tibble: 2 x 5
##    fips Name               .model lb_stat lb_pvalue
##   <int> <chr>              <chr>    <dbl>     <dbl>
## 1 37067 Forsyth County     PS        20.2    0.0273
## 2 37119 Mecklenburg County PS        20.5    0.0251
```

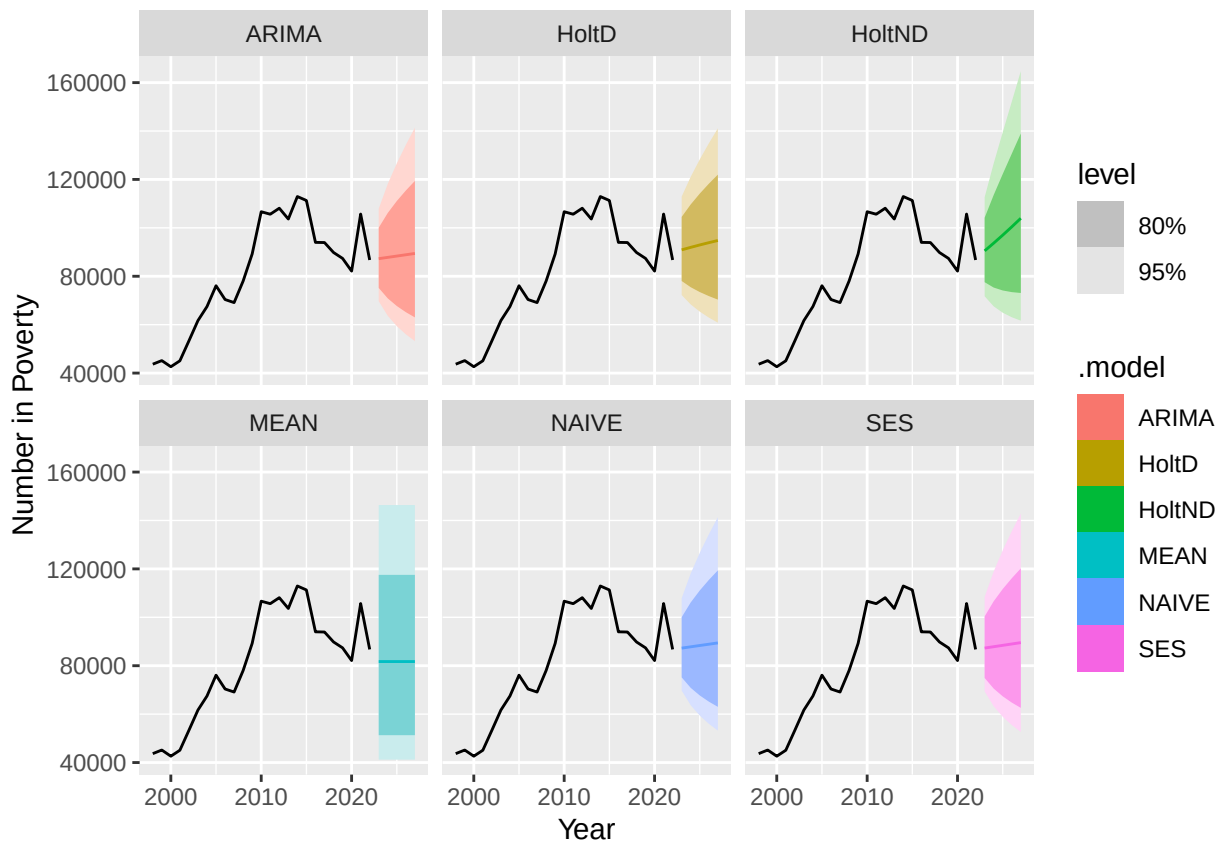
Two counties' residuals are significantly different from white noise.

The linear model does a good job of predicting the number in poverty since only two counties are significantly different from white noise.

Part 3: Stochastic Models

3.1 Single County Forecasts

```r
WC <- NCTS %>% filter(Name == topNine[1]) #Wake County

fitSetWC <- WC %>% model(NAIVE = NAIVE(log(`Number in Poverty`)),
                         MEAN = MEAN(log(`Number in Poverty`)),
                         SES = ETS(log(`Number in Poverty`) ~ error("A") + trend("N") + season("N")),
                         HoltD = ETS(log(`Number in Poverty`) ~ error("A") + trend("Ad") + season("N"))
                         HoltND = ETS(log(`Number in Poverty`) ~ error("A") + trend("A") + season("N"))
                         ARIMA = ARIMA(log(`Number in Poverty`)))

fitSetWC %>% forecast(h = "5 years") %>% autoplot(WC) + facet_wrap(".model")
```




```r
fitSetWC %>% accuracy() %>% group_by(.model) %>% summarize(RMSE = mean(RMSE))
```

```
## # A tibble: 6 x 2
##   .model   RMSE
##   <chr>   <dbl>
## 1 ARIMA   9346.
## 2 HoltD   8881.
## 3 HoltND  9400.
## 4 MEAN   22784.
## 5 NAIVE   9539.
## 6 SES     9346.
```

Based on the RMSE scoring, the Holt's additive linear damped method smoothing is the best for this county.

3.2 Exponential Smoothing Models

```
fitSetES <- NCTS %>%
  model(SES = ETS(log(`Number in Poverty`) ~ error("A") + trend("N") + season("N")),
        HoltD = ETS(log(`Number in Poverty`) ~ error("A") + trend("Ad") + season("N")),
        HoltND = ETS(log(`Number in Poverty`) ~ error("A") + trend("A") + season("N"))
        )

fitSetES %>% glance() %>% group_by(.model) %>% summarize(AICc = mean(AICc))
```

```
## # A tibble: 3 x 2
##   .model  AICc
##   <chr>  <dbl>
## 1 HoltD  -22.2
## 2 HoltND -24.6
## 3 SES    -29.8
```

Based on the AICc score, the simple exponential smoothing model is best since it has the lowest AICc.

3.3 ARIMA Models

```
ARIMAs <- NCTS %>% model(AA = ARIMA(log(`Number in Poverty`)))
ARIMAs %>% mutate(Type = ARIMAs$AA %>% as.character()) %>% group_by(Type) %>% summarise(Amount = n())
```

```
## # A tibble: 7 x 2
##   Type                    Amount
##   <chr>                    <int>
## 1 <ARIMA(0,0,0) w/ mean>       2
## 2 <ARIMA(0,1,0)>              40
## 3 <ARIMA(0,1,1)>               3
## 4 <ARIMA(1,0,0) w/ mean>      44
## 5 <ARIMA(1,0,2) w/ mean>       1
## 6 <ARIMA(1,1,0)>               8
## 7 <ARIMA(2,0,0) w/ mean>       2
```

The most commonly selected models are ARIMA(1,0,0) w/ mean and ARIMA(0,1,0) (random walk).

```
fitSetA <- NCTS %>% model(A010 = ARIMA(log(`Number in Poverty`) ~ pdq(0,1,0)),
                          A100M = ARIMA(log(`Number in Poverty`) ~ pdq(1,0,0) + 1))

fitSetA %>% glance() %>% group_by(.model) %>% summarize(AICc = mean(AICc))
```

```
## # A tibble: 2 x 2
##   .model  AICc
##   <chr>  <dbl>
## 1 A010   -39.9
## 2 A100M  -38.3
```

Based on the AICc evaluator, the ARIMA(0,1,0) (random walk) model is best since it has the lowest AICc.

When Including +1 to the ARIMA(1,0,0) with mean model, the results did not change. After some research I found that pdq(1,0,0) internally includes the + 1 intercept.

3.4 Cross Validation

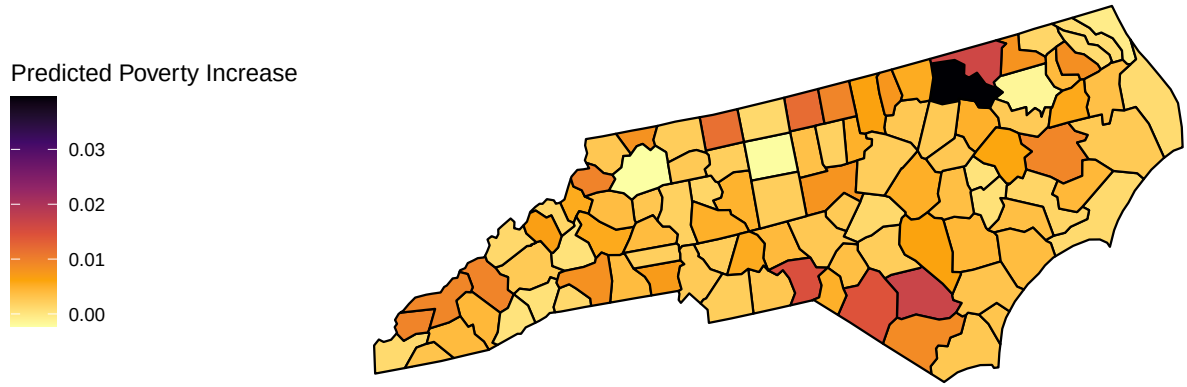
```r
finalfit <- NCTS %>% stretch_tsibble(.init = 20) %>% model(
  SES = ETS(log(`Number in Poverty`) ~ error("A") + trend("N") + season("N")),
  A010 = ARIMA(log(`Number in Poverty`) ~ pdq(0,1,0)))

fff <- finalfit %>% forecast(h = "5 years")

fff %>% accuracy(NCTS) %>% select(.model, fips, Name, .type, RMSE) %>% group_by(.model) %>% summarize(RM
```

```
## # A tibble: 2 x 2
##   .model    RMSE
##   <chr>    <dbl>
## 1 A010   198512.
## 2 SES    178377.
```

Based on the summed RMSE, the Simple Exponential Smoothing model preformed the best on cross validation.

Part 4: Forecasts

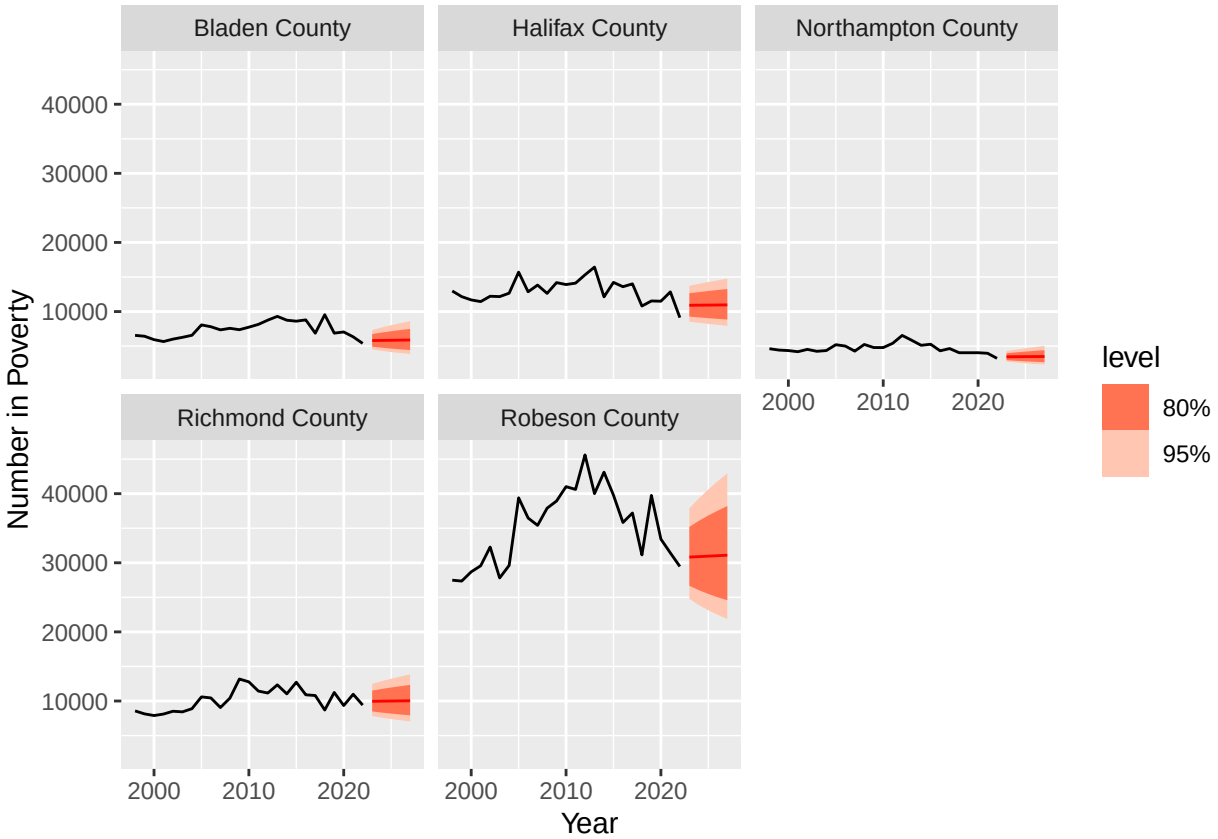
```r
bestFit <- NCTS %>% model(ETS(log(`Number in Poverty`) ~ error("A") + trend("N") + season("N")))

bff <- bestFit %>% forecast(h = "5 years")

prediction <- bff %>% filter(Year == 2027) %>% pull(.mean)

NorthCarolina <- NCTS %>% filter(Year == 2022) %>%
  mutate(Prediction = prediction,
         "Percentage Increase" = (prediction - `Number in Poverty`)/`County Population`,
         "Poverty Increase" = (Prediction - `Number in Poverty`)) %>%
  select(c(Year, fips, Name, `Number in Poverty`, Prediction, `Poverty Increase`, `Percentage Increase`)
  rename(values = `Poverty Increase`)

plot_usmap(regions="counties",include="North Carolina", data = NorthCarolina, values = "Percentage Incre
  labs(fill = "Predicted Poverty Increase") + scale_fill_viridis_c(option = "B", direction = -1) + theme
```




```
NorthCarolina %>% rename(`Poverty Increase` = values) %>% arrange(desc(`Percentage Increase`)) %>%
  select(-`Poverty Increase`) %>% head(5)
```

```
## # A tsibble: 5 x 6 [1Y]
## # Key:       fips, Name [5]
##    Year  fips Name          `Number in Poverty` Prediction `Percentage Increase`
##   <int> <int> <chr>                       <dbl>      <dbl>                 <dbl>
## 1  2022 37083 Halifax Coun~                9123     10965.                0.0397
## 2  2022 37017 Bladen County                5392      5884.                0.0169
## 3  2022 37131 Northampton ~                3229      3498.                0.0164
## 4  2022 37153 Richmond Cou~                9410     10034.                0.0150
## 5  2022 37155 Robeson Coun~               29466     31099.                0.0145
```

```
#Note that removing the `Poverty Increase` column is purely for visual purposes.

NC5 <- NorthCarolina %>% arrange(desc(`Percentage Increase`)) %>% head(5) %>% pull(Name)

bff %>% filter(Name %in% NC5) %>% autoplot(NCTS, colour = "red") + facet_wrap("Name")
```




```r
write.csv(NorthCarolina, "Data/North_Carolina_Predicted_Poverty_Increase.csv")
```

We predict the that Halifax, Bladen, Northampton, Richmond, and Robeson County are the five counties that will have the highest percentage increase in poverty over the next five years.